Structured Programming

Part 1: Algorithms

By the end of this lesson, you should be able to:

- Explain what an algorithm is.
- Identify the three fundamental control structures used in structured programming.
- Explain how sequence, selection, and iteration change the flow of a program.

Before Structured Programming

Early programs often relied on the `GOTO` statement, which allowed the program to jump directly to any labeled line of code.

While this made simple programs possible, larger programs often became confusing because execution could jump unpredictably between many different locations.

In the 1960s and 1970s, structured programming replaced this approach by using sequence, selection (if statements), and iteration (loops), making programs easier to read, test, and maintain.

Structured Programming

Structured programming organizes programs using predictable control structures instead of jumping unpredictably between different sections of code.

The three fundamental control structures are:

Sequence — Perform instructions in order.

Selection — Choose which instructions to perform.

Iteration — Repeat instructions.

These structures make programs easier to read, test, and maintain.

Algorithms

An **algorithm** is a sequence of step-by-step instructions designed to perform a task or solve a problem.

Programs are implementations of algorithms using a programming language.

Algorithms often:

- Receive **input**.
- **Process** that information.
- Produce **output**.
- Use control structures to determine which steps happen and when.

Sequence

A **sequence** executes instructions one after another in the order they appear.

The program starts with one instruction, completes it, and then moves to the next.

A recipe is a real-world example of sequence because its instructions are followed step by step in order.

```
# First get an input from the user
name = input("What is your name? ")

# Second process the input data
message = "Hello " + name

# Lastly output the new message
print(message)
```

Selection

A **selection** allows a program to choose between different paths.

A condition is evaluated as either `True` or `False`, and the result determines which instructions are performed.

Python's `if` and `else` statements are examples of selection.

```
# Start with an input value
age = 16

# Different code will run
# depending on the value
if age >= 16:
    print("You can drive.")
else:
    print("You cannot drive yet.")
```

Iteration

An **iteration** repeats a set of instructions.

Instructions can repeat:

- A specific number of times.
- Until a particular condition is met.

Loops make programs more efficient because the same instructions do not need to be written repeatedly.

```
# This is an example
# of a for loop.

# This for loop will
# run 5 times.
for _ in range(5):
    print("Hello!")
```